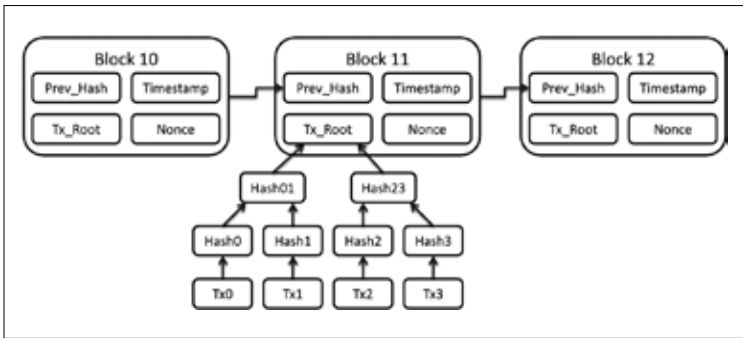


is then hashed along with the hash from the previous block (and a nonce-junk value). In Bitcoin's case the proof of work is to determine a nonce value that results in a certain number of zeros at the start of the resulting hash. Whoever finds it first adds their signature to the block and is 'awarded' some value, and the updated ledger is published so everyone can copy it and race to find the next block. In this way, the history of transactions is being recorded in a series of hashes. The slightest change to any detail of the past will ripple through and make every block after it look different. And since everyone on the network has a copy, unless you control more than half the network, you can't change history!



A schematic of how a Merkle tree is integrated into a blockchain

Let's say we have a hypothetical hash function that produces 64-bit hex codes. (This offers too few possibilities to be practically useful, but let's go with it for the sake of understanding.) We are using it to create a blockchain for the content of this FastTrack booklet, so that we don't have to check it for errors at multiple stages. We supply the first chapter of this FastTrack as input and obtain the digest - 8fa3. This string is appended to the second chapter and hashed to obtain the second digest - 4e32 (which represents the content of both chapter 1 and 2 uniquely). As we write this third chapter, we notice an extra question mark in the first chapter where it shouldn't be. Now we must go back, correct the text, and pass it to the hash function again. This time we obtain b402, even though only one character was changed. Similarly we redo the second chapter's hash with the correct hash from chapter one to obtain cd7c. What we just did is called a fork - we altered history because we were the only ones using the blockchain. On a practical distributed blockchain, this is NOT supposed to happen unless